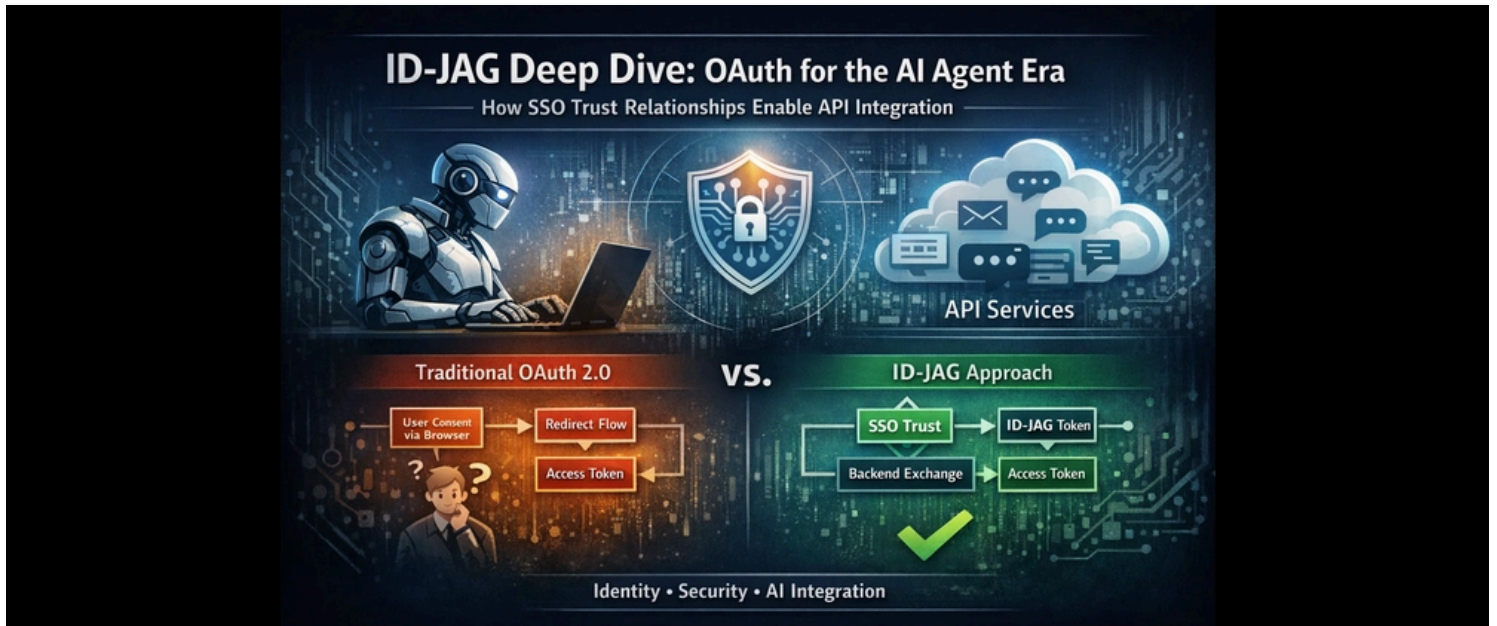




kt

Posted on Mar 16 • Edited on Apr 21



3

ID-JAG Deep Dive

#oauth #security #identity #ai

OAuth (9 Part Series)

- 1 RFC 6749 Deep Dive: Understanding OAuth 2.0 Design Decisio...
- 2 RFC 6750 Deep Dive: How Bearer Tokens Actually Work, Straig...
- ... **5 more parts...**
- 8 RFC 7636 Deep Dive: How PKCE Kills Authorization Code Interc...
- 9 RFC 7523 Deep Dive: JWT Profile

Introduction

Lately, while following discussions about AI agent architectures and integration patterns, I keep seeing the keyword "ID-JAG" pop up.

When an LLM agent calls an external SaaS API on behalf of a user, the biggest wall it hits is the authorization flow. Traditional OAuth requires "redirecting the user to a browser to get consent," but this fundamentally doesn't work for AI agents. Agents don't have a browser UI, and you can't expect an autonomous process to wait for a manual consent click.

How do you achieve secure cross-domain API access without user interaction (no-consent)?

ID-JAG (Identity Assertion JWT Authorization Grant) is the IETF's new approach to this problem.

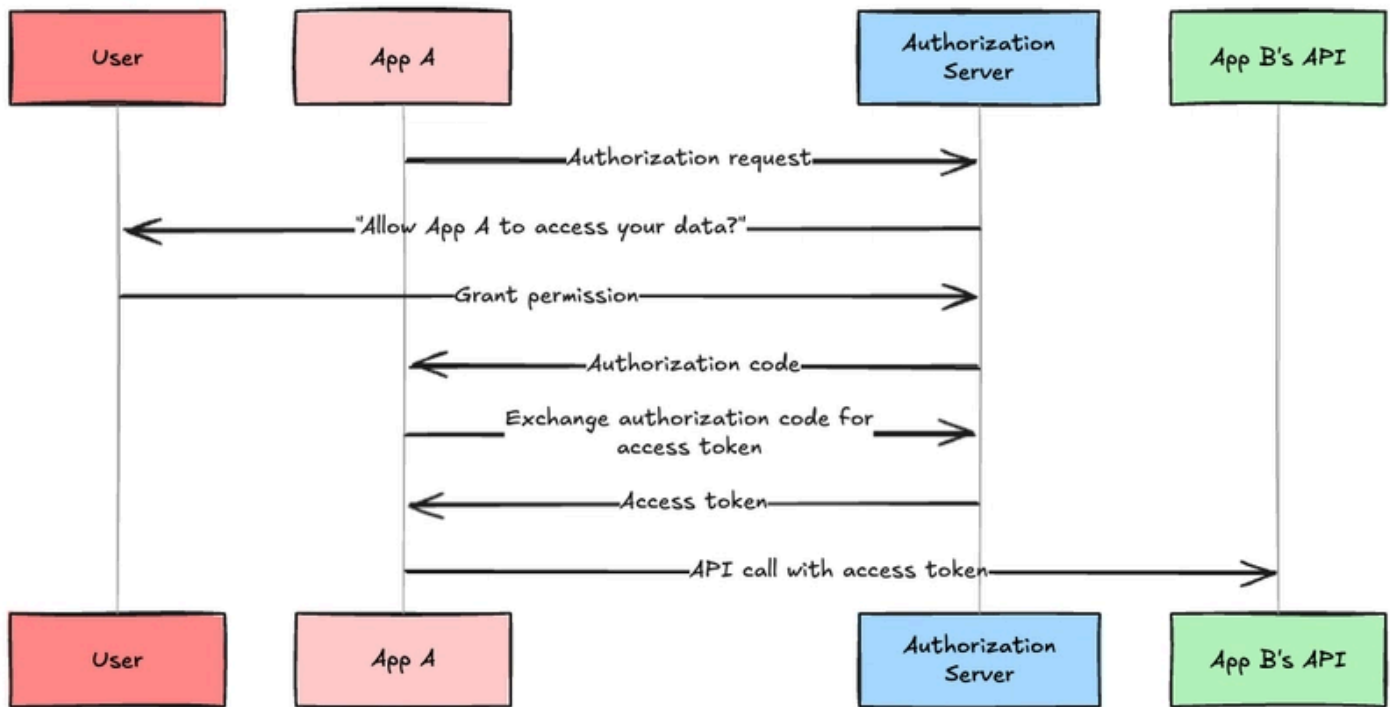
The core idea is simple: **"Extend the trust relationship already established through SSO with an IdP (Identity Provider) to API integration as well."**

Background: The Relationship Between OAuth 2.0 and SSO

To understand ID-JAG, you first need to understand the relationship between OAuth 2.0 and SSO (Single Sign-On).

OAuth 2.0 Refresher

OAuth 2.0 is a framework for "one app accessing another app's API on behalf of a user."



The key point is that there's a **"user grants consent via browser"** step. OAuth 2.0's security model depends on this user consent.

SSO and IdPs

SSO (Single Sign-On) lets users log into multiple apps with a single account.

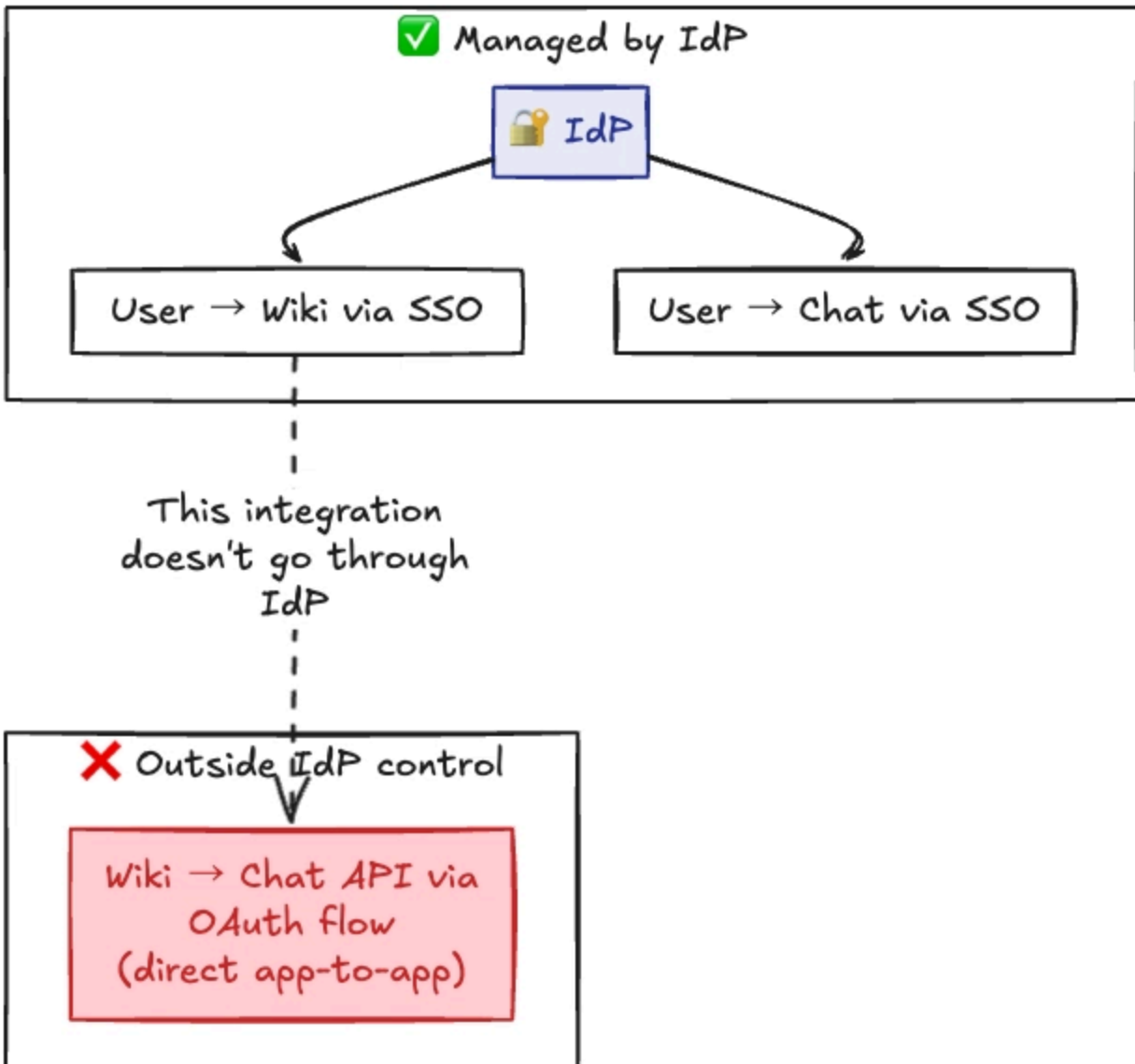
Enterprises use IdPs (Identity Providers) like Okta, Microsoft Entra ID (formerly Azure AD), or Google Workspace.

When logging in via SSO, the IdP issues an **ID Token** (in OpenID Connect) or a **SAML Assertion**. Think of it as a certificate that says "this user is who they claim to be."

The Problem: The Gap Between SSO and API Access

This is the heart of the matter.

Enterprise IT admins can centrally manage "which users can log into which apps" through SSO configuration. But when it comes to **app-to-app API integration**, that control suddenly breaks down.



When Wiki wants to access Chat's API, traditional OAuth 2.0 requires:

1. Redirect the user to Chat's authorization server
2. User grants consent via browser
3. Wiki gets an access token

This entire flow **completely bypasses the IdP**. Wiki and Chat are communicating directly. The enterprise IT admin can neither see nor control this integration.

Problem Summary

Problem	Impact
User experience	Consent screens pop up every time apps need to integrate

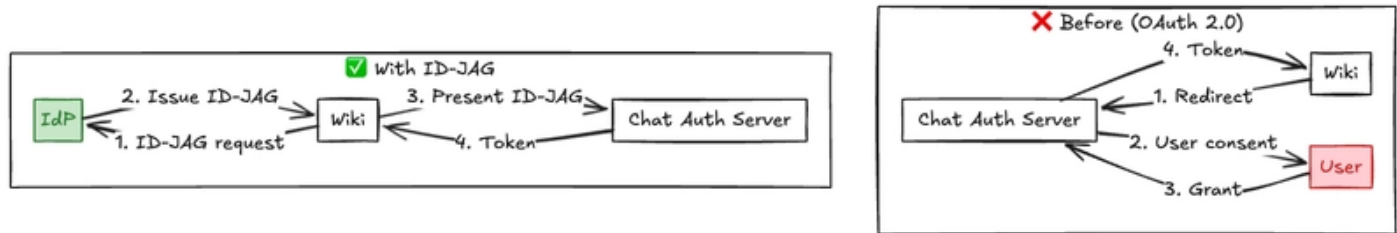
Problem	Impact
IT management	No visibility into which apps access which APIs
Policy enforcement	MFA requirements and IP restrictions set in IdP don't apply to API integrations
AI agents	Redirect flows are unusable for agents that have no browser

The Big Picture of ID-JAG

ID-JAG (Identity Assertion JWT Authorization Grant) is an IETF draft specification designed to solve the problems above (draft-ietf-oauth-identity-assertion-authz-grant , latest is draft-02).

In One Sentence

"Use the trust relationship already established with the IdP through SSO to obtain cross-domain access tokens without user interaction."



The left side (traditional) requires user browser interaction. The right side (ID-JAG) completes entirely on the backend.

The Cast of Characters

There are four actors in the ID-JAG flow. Rather than just parroting the RFC, let's use a concrete example.

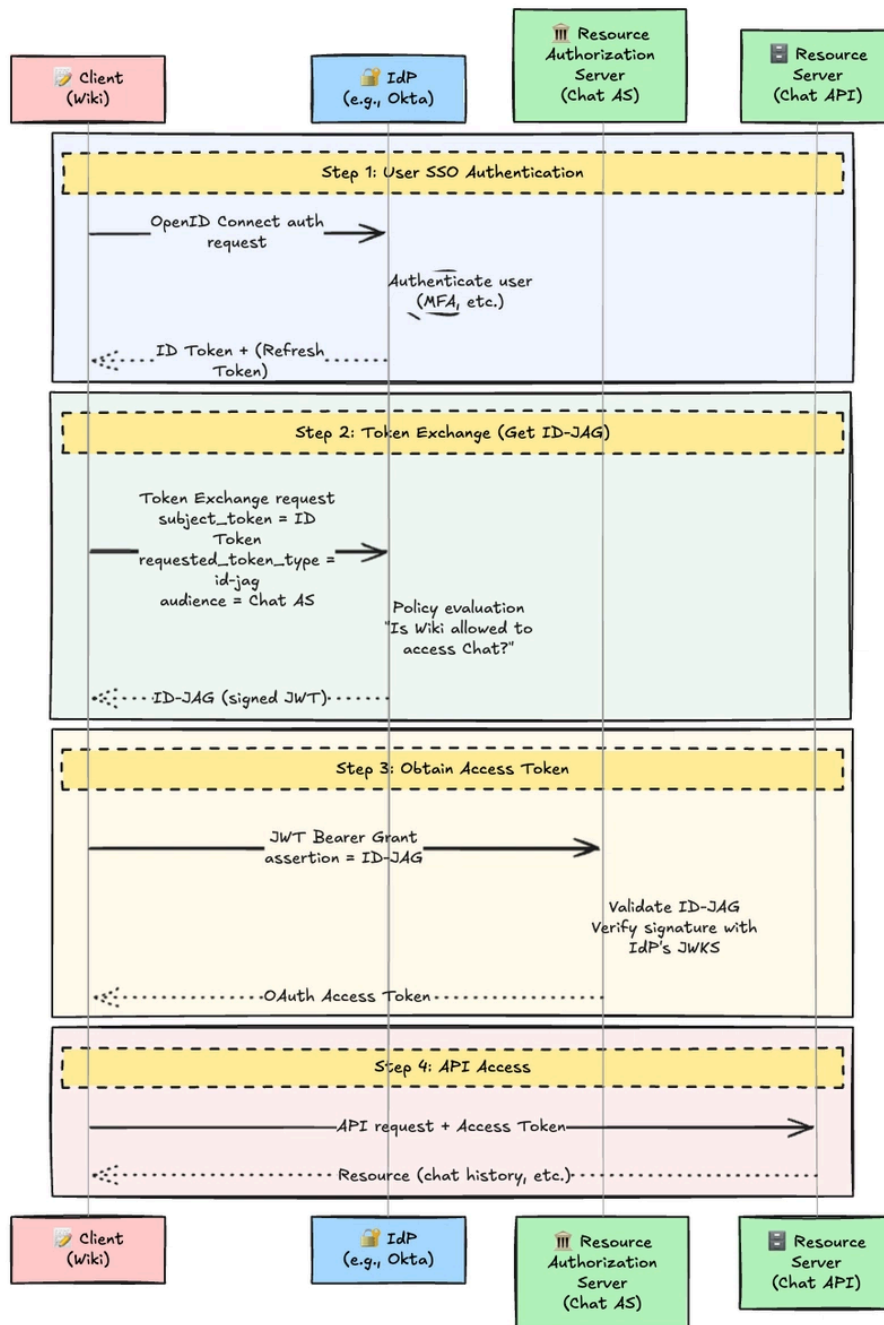
Role	Example	Responsibility
Client	 Wiki App	Wants to call another app's API on behalf of user
IdP Authorization Server	 Enterprise IdP (e.g., Okta)	Manages SSO and issues ID-JAGs
Resource Authorization Server	 Chat App's Auth Server	Validates ID-JAG and issues access tokens
Resource Server	 Chat API	Validates access tokens and returns resources

The critical point: **both the Client and the Resource Authorization Server have an established SSO trust relationship with the same IdP**. This shared trust is the prerequisite for ID-JAG.

The ID-JAG Flow in Detail

Now for the main event. Let's walk through the 4-step flow from top to bottom.

Full Sequence Diagram



Let's dig into each step.

Step 1: User SSO Authentication

This is a standard OpenID Connect flow — nothing ID-JAG-specific. When the user logs into the Wiki app, they're redirected to the enterprise IdP for authentication.

302 Redirect

Location: `https://acme.idp.example/authorize?response_type=code`


```
&scope=openid%20offline_access  
&client_id=wiki-app-client-id
```

The IdP authenticates the user (including MFA if configured) and returns an authorization code to the Wiki app, which exchanges it for an ID Token.

```
{  
  "id_token": "eyJraWQiOiJzMtZ0cVNtODhwREo4VGZCXzdrSEtQ...",  
  "token_type": "Bearer",  
  "access_token": "7SliwCQP1brGdjBtsaMnXo",  
  "refresh_token": "tGzv3J0kF0XG5Qx2TlKWIA",  
  "scope": "openid offline_access"  
}
```

The ID Token obtained here will be used in Step 2 as proof that "this user is who they claim to be." Requesting the `offline_access` scope to get a Refresh Token makes it easier to re-obtain the ID Token later.

Step 2: Token Exchange (Getting the ID-JAG)

This is the heart of ID-JAG.

The Wiki app sends a request to the IdP's **Token Exchange endpoint** saying "give me an ID-JAG for the Chat app." The protocol used is RFC 8693 (OAuth 2.0 Token Exchange).

What is Token Exchange (RFC 8693)? — A generic protocol for "presenting Token A to receive Token B." It was originally designed to support various exchange patterns like "ID Token → Access Token" or "Access Token → Access Token with different scope." ID-JAG uses the exchange pattern "ID Token (or Refresh Token) → ID-JAG." Note that the `actor_token/actor_token_type` parameters from the Token Exchange spec are not used in ID-JAG.

Request

```
POST /oauth2/token HTTP/1.1  
Host: acme.idp.example  
Content-Type: application/x-www-form-urlencoded
```



```
grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&requested_token_type=urn:ietf:params:oauth:token-type:id-jag
&audience=https://acme.chat.example/
&resource=https://api.chat.example/
&scope=chat.read+chat.history
&subject_token=eyJraWQiOiJzMTZ0cVNtODhwREo4VGZCXzdrSEtQ...
&subject_token_type=urn:ietf:params:oauth:token-type:id_token
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=eyJhbGciOiJSUzI1NiIsImtpZCI6IjIyIn0...
```

Breaking down each parameter:

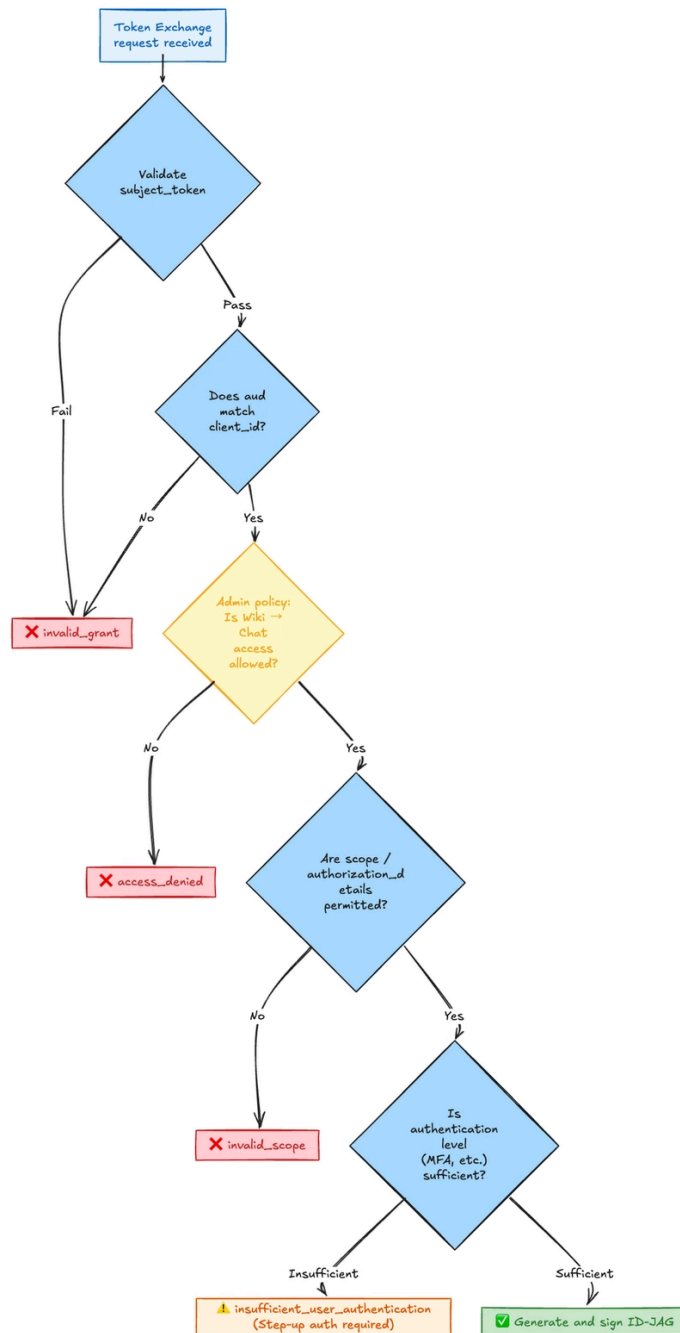
Parameter	Value	Meaning
grant_type	token-exchange	RFC 8693 Token Exchange
requested_token_type	id-jag	"Give me an ID-JAG"
audience	Chat AS URL	The intended recipient of the ID-JAG
resource	Chat API URL	The resource to be accessed
scope	chat.read chat.history	Required permissions
subject_token	ID Token	Proof of "on behalf of this user"

In draft-02, **Refresh Tokens** can also be used as the `subject_token`. Even if the ID Token has expired, a new ID-JAG can be obtained directly using the SSO Refresh Token. For SAML assertion environments, a path is also defined to first exchange the assertion for a Refresh Token and then obtain the ID-JAG.

```
// Using a Refresh Token
subject_token=tGzv3J0kF0XG5Qx2TLKWIA
&subject_token_type=urn:ietf:params:oauth:token-type:refresh_token
```

IdP Internal Processing

When the IdP receives this request, it performs the following processing:



The yellow step — **"Admin policy evaluation"** — is the biggest difference from traditional OAuth 2.0. Instead of the user clicking a consent button, **access is granted or denied based on policies pre-configured by the IT admin in the IdP.**

In draft-02, `authorization_details` (RFC 9396 Rich Authorization Requests) is also evaluated alongside `scope`. This enables fine-grained authorization (such as read access to a specific channel) that can't be expressed with simple scope strings.

Response

If access is granted, the IdP returns a signed ID-JAG.

```
{
  "issued_token_type": "urn:ietf:params:oauth:token-type:id-jag",
  "access_token": "eyJhbGciOiJIUzI1NiIsI...\"",
  "token_type": "N_A",
  "scope": "chat.read chat.history",
  "expires_in": 300
}
```

The `token_type` is `N_A` because this isn't an OAuth access token — it's an "assertion." It's placed in the `access_token` field due to historical reasons (RFC 8693 spec constraints). Don't let the field name mislead you.

Inside the ID-JAG

When you decode the issued ID-JAG, here's what the JWT looks like:

```
// Header
{
  "alg": "ES256",
  "typ": "oauth-id-jag+jwt"
}
// Payload
{
  "jti": "9e43f81b64a33f20116179",
  "iss": "https://acme.idp.example/",
  "sub": "U019488227",
  "aud": "https://acme.chat.example/",
  "client_id": "f53f191f9311af35",
  "exp": 1311281970,
  "iat": 1311280970,
  "resource": "https://api.chat.example/",
  "scope": "chat.read chat.history",
  "auth_time": 1311280970,
  "amr": ["mfa", "phrh", "hwk", "user"],
  "email": "alice@acme.example"
}
```

Here's what each claim means:

Claim	Required/Optional	Meaning
iss	Required	IdP identifier (who issued it)
sub	Required	User ID (for whom). Must match the identifier used during SSO with the Resource AS
aud	Required	Resource AS identifier (intended recipient)
client_id	Required	Client identifier (which app will use it)
jti	Required	Unique JWT ID (replay attack prevention)
exp / iat	Required	Expiration / Issued-at time
resource	Optional	Target API URL (single URI or URI array)
scope	Optional	Requested permissions
authorization_details	Optional	RAR authorization details (added in draft-02)
tenant	Optional	Tenant ID for multi-tenant issuers (added in draft-02)
aud_tenant	Optional	Resource AS tenant ID (added in draft-02)
aud_sub	Optional	User identifier at the Resource AS (added in draft-02)
cnf	Optional	Confirmation claim for DPoP key binding (jkt property contains JWK SHA-256 Thumbprint)

Claim	Required/Optional	Meaning
email	Optional (recommended)	User email address (for account resolution)
auth_time / acr / amr	Optional	Authentication time / Authentication context / Authentication methods

In draft-02, including `email` and `aud_sub` is **recommended**. This is important for the Resource AS to resolve the user's account — especially for **JIT (Just-In-Time) account creation** for users who haven't SSO'd yet.

Difference from ID Tokens

You might be thinking "how is this different from an ID Token?" They look almost identical, but there's a crucial difference.

Aspect	ID Token	ID-JAG
typ	None (or JWT)	oauth-id-jag+jwt
aud	Client app (self)	Resource Authorization Server
client_id	None	Client's client_id
Purpose	Tell the Client that user authentication is complete	Prove that the Client may access the API on behalf of the user
Consumer	Client (Relying Party)	Resource Authorization Server

The `aud` (audience) is the decisive difference. An ID Token says "To Wiki App: this user is authentic." An ID-JAG says "To Chat's Authorization Server: Wiki App is allowed to access the API on behalf of this user."

You must never pass an ID Token directly to another domain's authorization server. Per the OpenID Connect spec, only the Relying Party specified in the `aud` claim may receive and process an ID Token. ID-JAG exists precisely to work around this constraint.

Step 3: Obtaining the Access Token

Once the Wiki app has the ID-JAG, it presents it to Chat's authorization server (Resource Authorization Server) Token endpoint. The protocol used is RFC 7523 (JWT Bearer Grant).

```
POST /oauth2/token HTTP/1.1
Host: acme.chat.example
Authorization: Basic yZS1yYW5kb20tc2VjcmV0v3J0kF0XG5Qx2

grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
&assertion=eyJhbGciOiJIUzI1NiIsI...
```

Resource Authorization Server Validation

The Resource AS performs the following validations on the received ID-JAG:

1. **JWT typ check:** Must be `oauth-id-jag+jwt`
2. **aud validation:** Must match the Resource AS's own issuer identifier. If an array, it must contain exactly one element
3. **Signature verification:** Signature must be valid using the IdP's JWKS (public key set)
4. **client_id validation:** Must match the requesting client's authentication
5. **Expiration check:** `exp` must be valid, `jti` must not have been seen before
6. **Authorization info processing:** Evaluate `scope`, `resource`, and `authorization_details` against policy

Since the SSO trust relationship is already established, the location to fetch the IdP's public keys is already configured.

Response

```
{
  "token_type": "Bearer",
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "expires_in": 86400,
  "scope": "chat.read chat.history"
}
```

Finally, we have a standard OAuth access token.

Step 4: API Access

The rest is a standard API call.

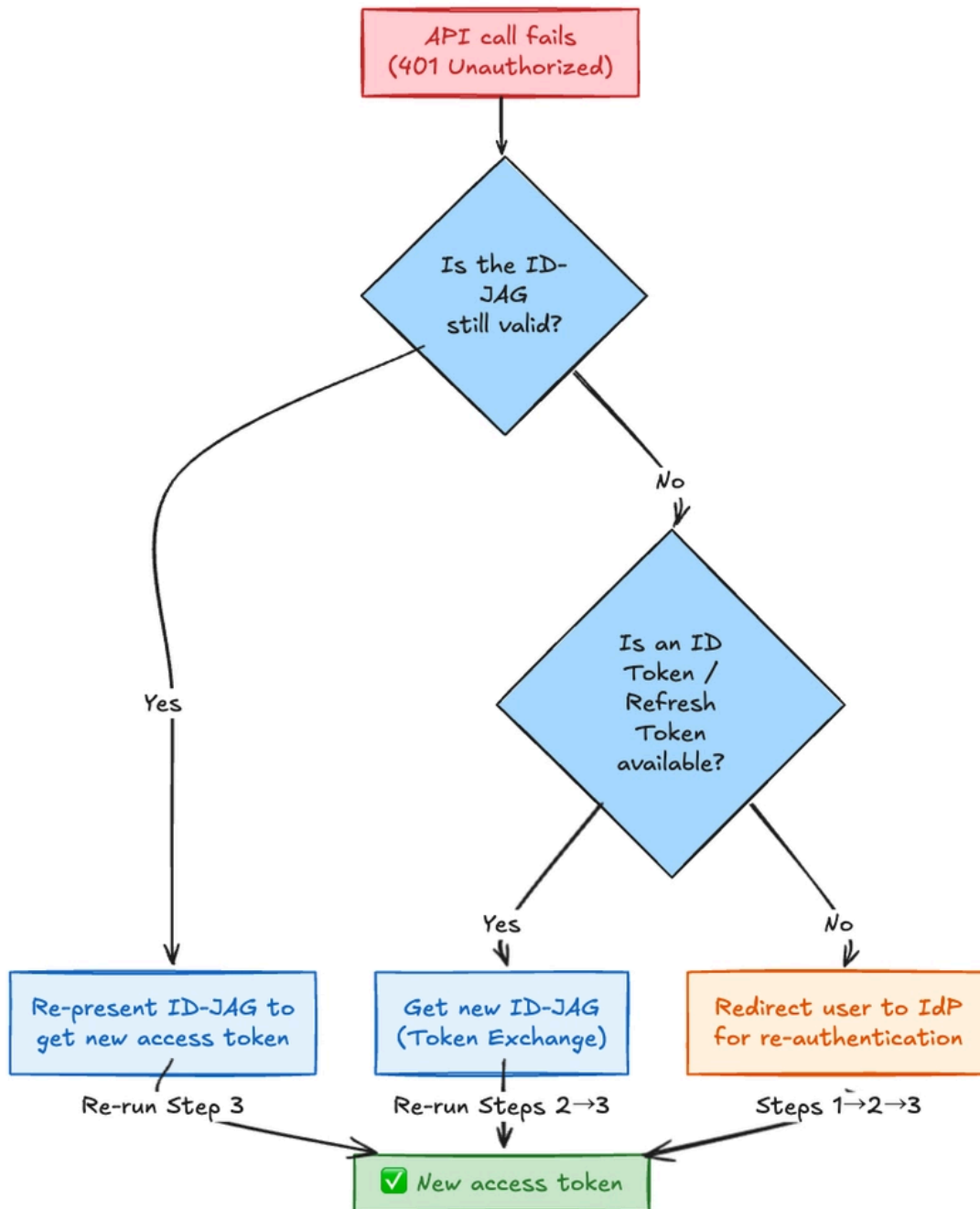
```
GET /api/channels/general/history  
Host: api.chat.example  
Authorization: Bearer 2YotnFZFEjr1zCsicMWpAA
```

Nothing ID-JAG-specific here. It's the same final step as any OAuth flow.

Token Renewal Flow

"What happens when the access token expires?"

In traditional OAuth 2.0, you'd use a Refresh Token. With ID-JAG, it's a bit different.



The draft spec **discourages issuing Refresh Tokens** from the Resource Authorization Server. Since you can re-present the ID-JAG itself to get a new access token, the ID-JAG effectively serves as a Refresh Token substitute.

Draft-02 also officially supports using Refresh Tokens as the `subject_token`. Only when neither the ID Token nor Refresh Token is available does the user need to be redirected for re-authentication.

Rich Authorization Requests (RAR) Support

An important feature added in draft-02 is support for **RAR (Rich Authorization Requests, RFC 9396)**.

Traditional `scope` is just a space-delimited string (`chat.read chat.history`, etc.), but RAR's `authorization_details` enables more structured authorization requests.

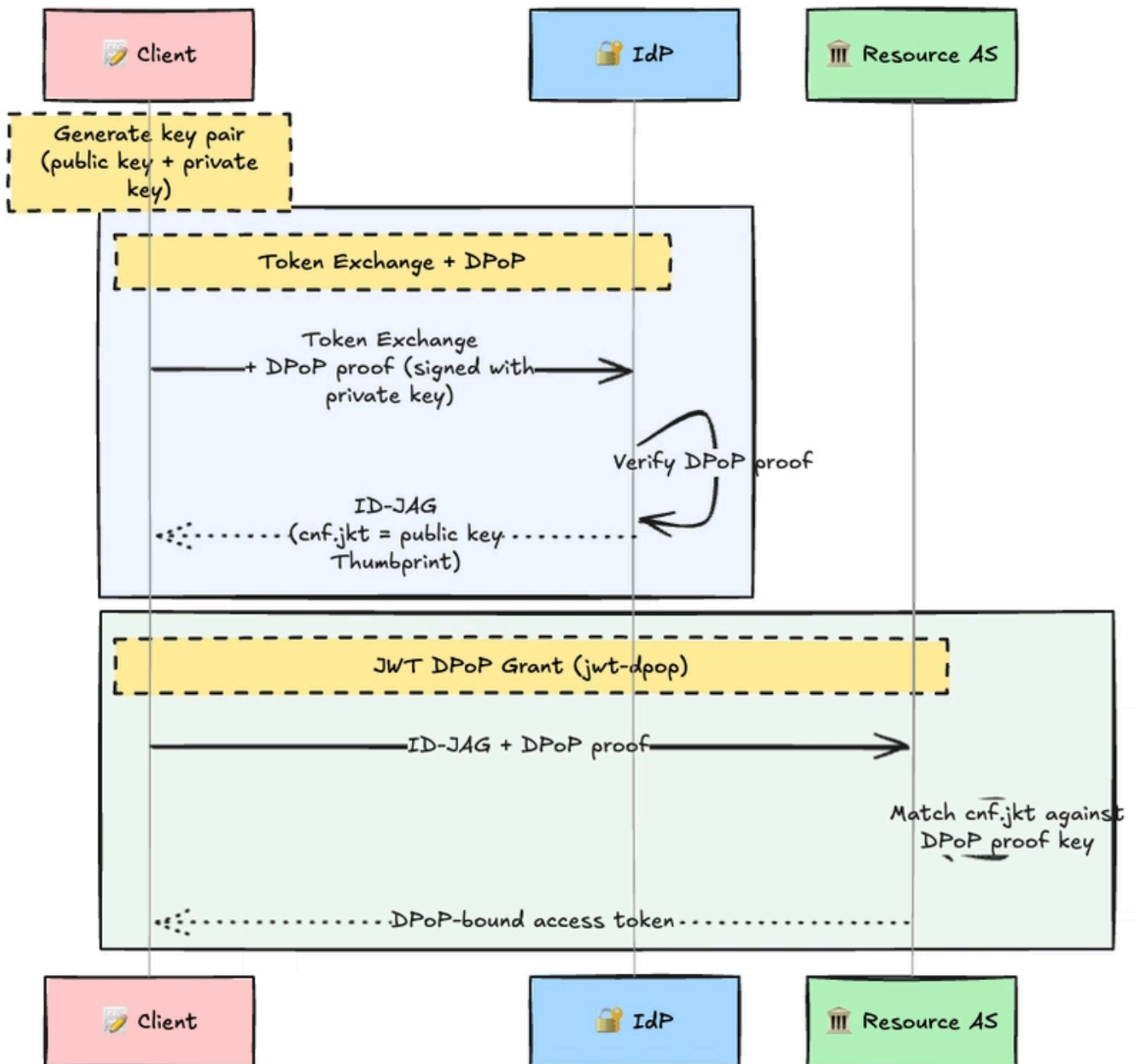
```
// Example authorization_details parameter in Token Exchange request
[
  {
    "type": "chat_read",
    "actions": ["read"],
    "locations": ["https://api.chat.example/channels"]
  },
  {
    "type": "chat_history",
    "actions": ["read"],
    "datatypes": ["message"]
  }
]
```

This lets you express fine-grained authorization like "read access to specific channels only" or "message data only." `scope` and `authorization_details` can be used together, with both the IdP and Resource AS evaluating and filtering each according to policy.

Sender-Constrained Tokens (DPoP)

Another important addition in draft-02 is **DPoP (Demonstrating Proof of Possession, RFC 9449)** for sender-constraining tokens.

A regular Bearer Token can be used by anyone who possesses it. With DPoP, **only the client holding the specific key pair's private key can use the token.**



When presenting an ID-JAG with a DPoP proof to the Resource AS, the `grant_type` is not the usual `jwt-bearer` but `urn:ietf:params:oauth:grant-type:jwt-dpop` (defined in `draft-parecki-oauth-jwt-dpop-grant`).

Downgrade attack mitigation: The client SHOULD verify that the ID-JAG returned by the IdP contains a `cnf` claim. If the IdP silently ignores the DPoP proof and returns an ID-JAG without `cnf`, the client can detect this. This prevents downgrade attacks where the IdP silently drops DPoP.

Rules when the ID-JAG contains a `cnf` claim:

ID-JAG has <code>cnf</code>	DPoP proof presented	Result
✓	✓	Verify thumbprint match → Issue DPoP-bound token (<code>grant_type = jwt-dpop</code>)
✓	✗	✗ <code>invalid_grant</code> error (proof required)
✗	✓	DPoP-bound token may be issued (at AS discretion)
✗	✗	Regular Bearer token issued (if AS allows; <code>invalid_grant</code> if AS requires sender-constrained tokens)

The Cross-Domain Client ID Problem

Here's an easy-to-overlook but critical issue.

The ID-JAG flow involves three independent authentication relationships:

- ① **Client → IdP** (OIDC/SAML): `client_id = wiki-at-idp`
- ② **Client → Resource AS** (OAuth): `client_id = wiki-at-chat`
- ③ **Resource AS → IdP** (OIDC/SAML): `client_id = chat-at-idp`

The Wiki app is registered with the IdP as `wiki-at-idp`. Meanwhile, it's registered with Chat AS as `wiki-at-chat`. **These are different `client_ids`.**

The `client_id` claim in the ID-JAG needs to contain the `client_id` that Chat AS recognizes (`wiki-at-chat`). But the IdP is the one issuing the ID-JAG. This means **the IdP needs to know what Wiki app's `client_id` is at Chat AS.**

Similarly, the `sub` claim requires attention. The same user may have different Pairwise Subject Identifiers for SSO with the Client vs. SSO with the Resource AS. The IdP must put the **subject identifier that the Resource AS recognizes** into the ID-JAG.

Two approaches address this:

1. **IdP maintains a mapping table:** The IdP manages the client_id and sub correspondence between each Client↔Resource AS (configured out-of-band)
2. **Client ID Metadata Document:** Use a document URL as the client ID to create a global namespace (draft-ietf-oauth-client-id-metadata-document)

Multi-Tenant Considerations

Draft-02 provides detailed definitions for the relationships between issuer , tenant , client_id , and sub in multi-tenant environments (RFC Section 6).

In enterprise environments, both IdPs and SaaS apps are often multi-tenant. The same IdP platform serves multiple companies (tenants).

Single-Tenant vs Multi-Tenant Issuer

Property	Single-Tenant Issuer	Multi-Tenant Issuer
issuer	https://idp.company.com	https://acme.okta.com
tenant claim	Not needed	Required
sub uniqueness	Unique by iss + sub	Unique by iss + tenant + sub
Example	On-premises IdP	Okta, Azure Entra ID

Single-Tenant example:

```
{
  "iss": "https://idp.company.com",
  "sub": "user@company.com",
  "aud": "https://app.example.com"
}
```

Multi-Tenant example:

```
{
  "iss": "https://acme.okta.com",
  "tenant": "tenant-123", // Tenant identifier
  "sub": "user@company.com",
}
```

```
"aud": "https://app.example.com"
}
```

Client ID and Tenant Relationships

In multi-tenant environments, there are two patterns for ensuring client_id uniqueness:

Pattern	Description	Example
Tenant-Scoped	client_id is unique only within a specific tenant. The same client_id may exist in different tenants	Unique by iss + tenant + client_id
Global	client_id is unique across all tenants. No duplicates across the entire IDP or Resource AS	Unique by client_id alone

Tenant-Scoped example:

```
IdP (Okta) tenant-123: client_id = "app-001"
IdP (Okta) tenant-456: client_id = "app-001" (same value is OK)
```

Global example:

```
IdP (Okta) across all tenants: client_id = "app-tenant-123-001" (unique across all tenants)
```

Resource AS-Side Claims in Multi-Tenant Environments

When the Resource AS itself is multi-tenant, additional claims come into play:

Claim	Description	Example
aud_tenant	Resource AS tenant identifier	res-tenant-456
aud_sub	User identifier at the Resource AS (for JIT account creation)	ACME_USER_789

When the IdP and Resource AS use different tenancy models, careful mapping is required.

```
// ID-JAG generated by IdP (multi-tenant aware)
{
```

```
"iss": "https://acme.okta.com",
"tenant": "tenant-123",           // IdP-side tenant
"sub": "user@company.com",
"aud": "https://resource-as.example.com",
"aud_tenant": "res-tenant-456",  // Resource AS-side tenant
"aud_sub": "ACME_USER_789",      // User ID at Resource AS
"client_id": "wiki-app"
}
```

SAML 2.0 Interoperability

Many enterprise environments still use SAML 2.0 as their SSO foundation. Draft-02 defines a bridge path from SAML to ID-JAG.

1. Send the SAML assertion as the `subject_token` in Token Exchange
2. Set `subject_token_type` to `urn:ietf:params:oauth:token-type:saml2`
3. First obtain a Refresh Token (`requested_token_type=refresh_token` , `scope=openid offline_access`)
4. Then use the Refresh Token as the subject to obtain an ID-JAG

The IdP maps the SAML assertion's Audience (SPEntityID) to an OAuth Client ID and validates it against the authenticated client.

Enterprise Deployment Use Case (RFC A.1)

The most common ID-JAG usage scenario. All enterprise employee applications (Wiki, Chat, CRM, etc.) are managed by the same enterprise IdP.

Prerequisites:

- Multiple SaaS applications configured with SSO through the same enterprise IdP
- Federation established between the IdP and each app's Resource AS
- IT admin defines "who can access what" policies for each app-to-app relationship

Benefits:

- **User experience:** Seamless app-to-app integration without consent screens

- **IT management:** Centrally managed from the IdP. Cross-app integration policies controlled in one place
 - **Security:** MFA and access audit logs centralized
 - **Compliance:** Changes to user permissions immediately reflected across all apps
-

Email/Calendaring Use Case (RFC A.2)

Automatic integration between multiple communication/collaboration tools (email, calendar, scheduling).

Scenario:

- A meeting invite received in the email app automatically syncs to a separate vendor's calendar app
- The calendar app automatically updates the user's status based on scheduled meetings

Prerequisites:

Unlike Enterprise Deployment, this is a **no pre-registration** scenario:

- The Client does **not have a pre-registered OAuth client** with either the IdP or the Resource AS
- The Client has already obtained an Identity Assertion (ID Token, etc.) from the IdP
- The Resource AS is **configured to accept ID-JAGs from unregistered clients**

Implementation with ID-JAG:

User SSO Login

→ IdP issues ID Token

Mail App needs Calendar API

→ Mail App requests ID-JAG from IdP

(subject_token = ID Token, audience = Calendar AS)

Calendar AS

→ validates ID-JAG, issues access token

→ Mail App syncs calendar events

Benefits:

- Users don't need to log in separately to each tool
 - Automatic sync between tools improves productivity
 - IT admins maintain unified permission management
-

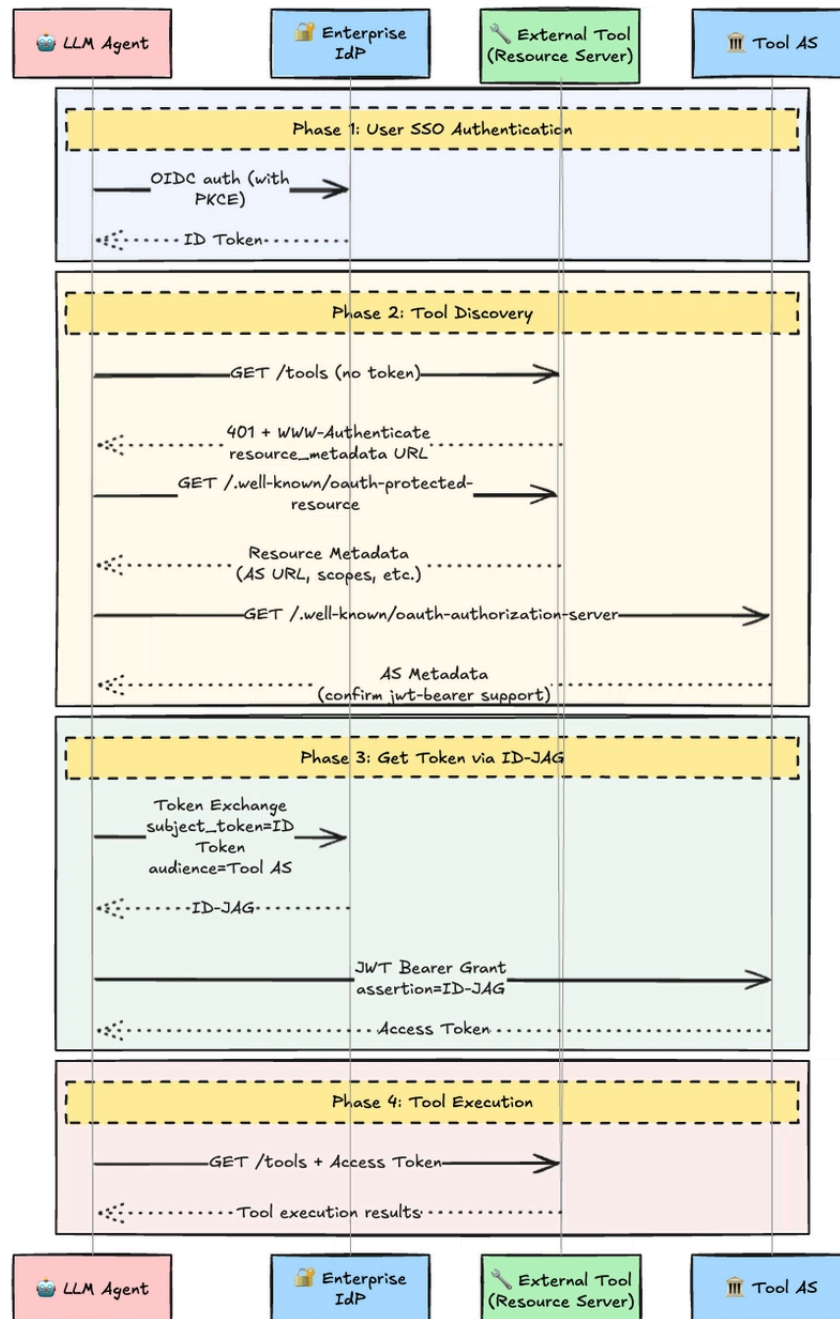
The 2026 Use Case: AI Agents

The most interesting use case appears in the draft spec's Appendix: **LLM agents accessing enterprise external tools.**

Prerequisites:

- The LLM Agent is registered as an OAuth client with the Enterprise IdP
- The LLM Agent is also registered as a client with the External Tool's Authorization Server
- The enterprise has established an SSO trust relationship between the IdP and the LLM Agent
- The enterprise has established an SSO + ID-JAG trust relationship between the IdP and the External Tool
- The enterprise has authorized the LLM Agent to access the External Tool on behalf of users with specific scopes

Full Flow for LLM Agents



The standout part here is **Phase 2: Tool Discovery**.

When the LLM agent accesses an external tool for the first time, it has no token. Making a raw API call returns 401 Unauthorized. The WWW-Authenticate header in the response contains the **Protected Resource Metadata** (RFC 9728) URL.

HTTP/1.1 401 Unauthorized

WWW-Authenticate: Bearer resource_metadata=

"https://saas.example.net/.well-known/oauth-protected-resource"

The agent can dynamically discover from this URL "which authorization server issues tokens for this API" and "what scopes are available." This is an **agent-native** authentication flow. No need to hardcode auth configuration for every API.

Furthermore, if the IdP's AS Metadata includes

`identity_chaining_requested_token_types_supported`, ID-JAG support can be dynamically detected.

Note: Even if the Resource AS metadata shows `jwt-bearer` support, that doesn't necessarily mean it accepts ID-JAG-format `jwt-bearer` assertions. `jwt-bearer` has other uses, so a dedicated metadata property indicating ID-JAG acceptance may be needed in the future. IdP-side discovery is solved by `identity_chaining_requested_token_types_supported` (former [issue #16](#), now closed), but the Resource AS-side discovery problem is still under discussion in [issue #34](#).

```
{
  "issuer": "https://cyberdyne.idp.example/",
  "grant_types_supported": [
    "authorization_code", "refresh_token",
    "urn:ietf:params:oauth:grant-type:token-exchange"
  ],
  "identity_chaining_requested_token_types_supported": [
    "urn:ietf:params:oauth:token-type:id-jag"
  ]
}
```

Why This Matters

With traditional approaches, allowing an AI agent to access external APIs means:

- Statically configuring API keys or Service Accounts for each API
- The agent holds powerful privileges (no per-user permission control)
- Prone to becoming "shadow AI" that IT admins can't track

With ID-JAG:

- Access tied to user permissions (the agent acts on behalf of the user)
- Access controlled by IdP policy (centrally managed by IT admins)
- Short-lived tokens (safer than static API keys)

Security Considerations

Key security requirements defined in RFC Section 8.

1. Confidential Clients Only (RFC 8.1)

ID-JAG should only be used with **Confidential Clients** (server-side apps that can securely hold client secrets). Public Clients (SPAs, mobile apps) cannot perform client authentication and should use traditional OAuth 2.0 authorization code flows (with interactive user consent).

Follow RFC 9700 (OAuth 2.0 Security BCP) best practices.

2. Step-Up Authentication (RFC 8.2)

The IdP can check whether the user's current authentication level meets the target resource's policy requirements when receiving a Token Exchange request. For example, viewing chat may not require MFA, but accessing confidential channels might.

When the authentication level is insufficient, an `insufficient_user_authentication` error is returned:

```
{
  "error": "insufficient_user_authentication",
  "error_description": "Subject doesn't meet authentication requirements",
  "max_age": 5
}
```

In this case, the Client redirects the user back to the IdP for Step-Up authentication (additional MFA, etc.).

3. Cross-Domain Use (RFC 8.3)

Critical rule: The IdP must not issue access tokens to itself for ID-JAGs it has issued.

ID-JAG is strictly for cross-domain authorization and must not be used to access the IdP's own resources. Violating this could lead to unintended privilege escalation.

4. Sender Constraining Tokens (RFC 8.4)

To address the limitations of bearer tokens, "sender constraining" — binding tokens to specific clients — is recommended. Draft-02 added **DPoP (Demonstrating Proof-of-Possession)** support (RFC 9449).

Sender constraining with DPoP:

1. Client generates a DPoP proof JWT (containing its public key)
2. Sends it via DPoP header to the IdP during Token Exchange
3. IdP embeds a `cnf` (confirmation) claim in the ID-JAG
4. Client also attaches a DPoP proof when presenting the ID-JAG to the Resource AS
5. Resource AS verifies the ID-JAG's `cnf` against the DPoP proof's key

This ensures that even if the token is intercepted, the attacker cannot use it without the corresponding private key.

Authorization Server Metadata (RFC Section 7)

Authorization Servers supporting ID-JAG define several metadata parameters. This allows clients to dynamically discover whether an AS supports ID-JAG and which token types are available.

IdP (Identity Provider) metadata example:

```
{
  "issuer": "https://acme.idp.example",
  "token_endpoint": "https://acme.idp.example/oauth2/token",
  "jwks_uri": "https://acme.idp.example/.well-known/jwks",
  "authorization_endpoint": "https://acme.idp.example/oauth2/auth",

  // ID-JAG specific metadata
  "token_endpoint_auth_methods_supported": [
    "client_secret_basic",
    "private_key_jwt"
  ],
  "grant_types_supported": [
    "authorization_code",
    "refresh_token",
    "urn:ietf:params:oauth:grant-type:token-exchange"
  ],
```

```
// Defined in the Identity Chaining spec (draft-ietf-oauth-identity-chaining)
"identity_chaining_requested_token_types_supported": [
  "urn:ietf:params:oauth:token-type:id-jag"
]
}
```

Resource Authorization Server metadata example:

```
{
  "issuer": "https://acme.chat.example",
  "token_endpoint": "https://acme.chat.example/oauth2/token",
  "jwks_uri": "https://acme.chat.example/.well-known/jwks",

  // Shows jwt-bearer support (but see note below)
  "grant_types_supported": [
    "authorization_code",
    "client_credentials",
    "urn:ietf:params:oauth:grant-type:jwt-bearer"
  ]
}
```

Note: Even if the Resource AS metadata includes `jwt-bearer` , it's not possible to determine whether it accepts ID-JAG format. A dedicated metadata property indicating ID-JAG support on the Resource AS side is being discussed in [issue #34](#).

The key point is that both the IdP and Resource AS **expose their trusted public key information (JWKS)**. The Resource AS periodically fetches the IdP's JWKS to verify ID-JAG signatures.

The RFC Stack Behind ID-JAG

ID-JAG is built on top of existing RFC specifications. It's not "new magic" — it's a **profile** that combines existing OAuth building blocks.

Specification	Role in ID-JAG
RFC 8693 (Token Exchange)	Step 2: ID-JAG request to IdP
RFC 7523 (JWT Bearer Grant)	Step 3: Presenting ID-JAG to Resource AS

Specification	Role in ID-JAG
OpenID Connect	Step 1: Obtaining the ID Token
RFC 8414 (AS Metadata)	IdP / Resource AS discovery
RFC 9728 (Protected Resource Metadata)	Resource Server discovery
RFC 7519 (JWT)	ID-JAG format
RFC 8707 (Resource Indicators)	resource parameter
RFC 8725 (JWT Best Practices)	Typed JWT (<code>oauth-id-jag+jwt</code>)
RFC 9396 (RAR)	authorization_details parameter (draft-02)
RFC 9449 (DPoP)	Sender-Constrained Tokens (draft-02)
draft-parecki-oauth-jwt-dpop-grant	DPoP JWT Bearer Grant (<code>jwt-dpop</code>)

It clearly defines "what" to exchange and "under what conditions" in Token Exchange, ensuring interoperability between different vendors.

Industry Trends (2026)

Okta's Moves

Okta (where ID-JAG co-author Aaron Parecki works) is advancing its ID-JAG implementation under the **Cross App Access (XAA)** brand. A developer sandbox environment (`xaa.dev`) is also publicly available.

Keycloak

Keycloak, the open-source identity provider, is also developing JWT Authorization Grant-related features. Check the latest Keycloak release notes for updates.

Identity Chaining

draft-ietf-oauth-identity-chaining, the parent specification of ID-JAG, defines a more general-purpose cross-domain authorization chaining mechanism. ID-JAG is positioned as an "enterprise SSO-specific profile" of this specification.

Conclusion

Technically, ID-JAG is a profile built on existing RFCs (Token Exchange + JWT Bearer Grant) — it doesn't invent new cryptographic techniques or new protocols. What it does is "extend the trust relationship already established through SSO to API integration." That's it.

But in the enterprise world where SSO and API integration have been disconnected, this "that's it" is a big deal. IT admins can centrally manage SaaS-to-SaaS integration from the IdP, users are freed from consent screen fatigue, and AI agents can call external APIs with permissions tied to the actual user.

With draft-02 adding RAR, DPoP, multi-tenant support, and SAML interoperability, the spec is getting close to production-ready. Okta's XAA brand implementation shows that real product integration is already underway.

It's still a draft, so the spec may evolve, but if you're looking into enterprise SaaS integration or AI agent deployment, this is a specification worth getting ahead of now.

References

ID-JAG Draft Specification:

- [Identity Assertion JWT Authorization Grant \(draft-02\) - IETF Draft](#)
- [OAuth Identity and Authorization Chaining Across Domains](#)
- [RFC 8693 - OAuth 2.0 Token Exchange](#)
- [RFC 7523 - JWT Profile for OAuth 2.0 Authorization Grants](#)
- [RFC 8414 - OAuth 2.0 Authorization Server Metadata](#)
- [RFC 9728 - OAuth 2.0 Protected Resource Metadata](#)
- [RFC 9396 - OAuth 2.0 Rich Authorization Requests](#)
- [RFC 9449 - OAuth 2.0 Demonstrating Proof of Possession \(DPoP\)](#)

- [RFC 9700 - Best Current Practice for OAuth 2.0 Security](#)
- [JWT DPoP Grant \(draft-parecki-oauth-jwt-dpop-grant\)](#)

Implementations and Tools:

- [Okta Cross App Access \(XAA\)](#)
- [Keycloak Release Notes](#)

OAuth (9 Part Series)

- 1 [RFC 6749 Deep Dive: Understanding OAuth 2.0 Design Decisio...](#)
- 2 [RFC 6750 Deep Dive: How Bearer Tokens Actually Work, Straig...](#)
- ... **5 more parts...**
- 8 [RFC 7636 Deep Dive: How PKCE Kills Authorization Code Interc...](#)
- 9 [RFC 7523 Deep Dive: JWT Profile](#)

Top comments (0) ↕

[Code of Conduct](#) • [Report abuse](#)



Mobile App Security Predictions in 2026: How You Can Stay Ahead of Threats and Attacks

The mobile app threat landscape is constantly changing, with attackers continuously evolving techniques. In 2026, staying one step ahead of attackers will be crucial. With Guardsquare, achieve comprehensive mobile app security without compromises.

[Read more](#)

**kt**

btw

JOINED

Jan 8, 2026

More from **kt**

Why Can We Use "Shorter" Keys?: Key Length vs Security Bits, the Real Story

#security #cryptography #beginners #todayilearned

Why I Built awesome-authorization: Mapping the World of Auth Engines onto a Single Page

#authorization #security #opensource #showdev

RFC 7523 Deep Dive: JWT Profile

#oauth #jwt #security #authentication

**MongoDB**

PROMOTED





The best way to build AI-powered apps.

[Start Building](#)



Build gen AI apps that run anywhere with MongoDB Atlas

MongoDB Atlas bundles vector search and a flexible document model so developers can build, scale, and run gen AI apps without juggling multiple databases. From LLM to semantic search, Atlas streamlines AI architecture. Start free today.

[Start Free](#)